Informe de diseño: base de datos clave-valor simple

'simple kv'

1. Estructura y modelo de las réplicas

Cada réplica es un proceso Erlang creado mediante `spawn`, cuyo ciclo `receive` procesa un mensaje a la vez de forma secuencial. Su estado interno comprende: un mapa de claves a `{Valor, Timestamp}` (o tombstones), la lista de PID de todas las réplicas del clúster (incluyéndose a sí misma), el parámetro de capacidad máxima α, y un contador `ValidSize` de entradas vigentes. Inicialmente el mapa esta vacío y el contador en cero.

Para garantizar una comunicación asíncrona segura y evitar la mezcla respuestas en el buzón del actor, toda solicitud enviada por un cliente (como `put`, `get` o `delete`) se asocia a una referencia única (`Ref`), lo que permite un emparejamiento exacto entre la petición y su respuesta correspondiente. A nivel de ciclo de vida, la API utiliza `spawn` en lugar de `spawn_link` para mantener los procesos independientes del hilo o cliente que los inicia; su finalización se controla de forma explícita enviando un mensaje `stop/1`, permitiendo detener nodos de manera singular sin afectar al resto del clúster.

En cuanto almacenamiento, los datos se guardan junto a timestamp correspondiente. Por simplicidad de diseño, las tombstones no se recolectan mediante un proceso de limpieza de memoria, por lo que el mapa puede crecer de forma indefinida, y se permite el uso del término `tombstone` como valor de usuario. Las tombstones (entradas marcadas como borradas) no computan para el límite de capacidad α.

Las operaciones de modificación se confirman al cliente inmediatamente después de aplicarse en la réplica local, propagándose de forma asíncrona hacia las demás replicas. De este modo, se asegura una consistencia eventual respaldada por el modelo de intercambio de mensajes y las timestamps.

2. Protocolo de comunicación

En la tabla, R es la réplica destinataria, C el proceso cliente, V otra réplica, K una clave, X un valor, T una marca, D un TTL en milisegundos y r una referencia. Los mensajes de respuesta se identifican por su forma; los envíos son asíncronos. La tabla incluye también configuración, temporizador y cierre.

Mensaje y contenido	Emisor → receptor	Funcionalidad y efecto en el receptor
`{set_neighbor_threads, Pids}`	Inicializador → R	Sustituye la lista de replicas vecinas. Su funcionalidad principal inicializar esta lista para cada thread.
`{put, K, X, D, C, r}`	$C \to R$	Crea clave-valor local, guarda, responde `{r, ok}`, programa TTL, difunde `silent_put` y aplica desalojo local por α si es necesario.
`{silent_put, K, X, T}`	$R \to V$	Funcion de propagacion de 'put'. Acepta o rechaza la versión por timestamp; recalcula el contador, sin retransmitir ni desalojar. No contiene TTL.
`{delete, K, C, r}`	$C \to R$	Define una tombstone con timestamp nuevo en la clave, responde `{r, ok}` y propaga este cambio con `silent_delete`.
`{ttl_expired, K}`	Temporizador → R	Misma funcionalidad que 'delete', la diferencia es que esta no devuelve ningun valor.
`{silent_delete, K, T}`	$R \to V$	Funcion de propagacion de 'delete'. Propaga el borrado de una clave, el 'silent' implica que no devuelve nada al que envio el mensaje.
`{offer_key_value, K, X, T}`	$R \to V$	Funcion de ofrecimiento de clave-valor luego del desalojo local de la clave por α en alguna replica. La acepta si la timestamp supera la local o no hay entrada y si ValidSize $<\alpha$, no retransmite.
`{get, K, C, r}`	$C \to R$	Devuelve el valor de la clave guardada; si falta la clave localmente, intenta consultar a vecinos. Responde `{r, Result}` o `{r, not_found}` si no la encuentra.
`{get_no_propagate, K, R, q}`	$R \to V$	Funcion utilizada para buscar una clave en otras replicas, si es que localmente la inicial no la encuentra. Busca la clave solo localmente y devuelve el valor o `not_found`.
`{q, Result}`	$V \to R$	Una Replica responde a otra con el resultado de una consulta: el valor encontrado o `not_found`. La referencia q permite identificar a que consulta corresponde. `{ok, X}` o `not_found` sin referencia; tampoco coincide.
`{r, ok}`; `{r, Result}`	$R \to C$	Una replica responde al cliente que solicito la operacion. Devuelve ok para confirmar 'put' o 'delete', o el valor si es un 'get'. Nuevamente la referencia r identifica la solicitud.
`{stop, C}`; `{stopped, R}`	$C \to R$; $R \to C$	El cliente le pide a la replica que termine su ejecucion. Esta confirma con `{stopped, R}` y finaliza; el cliente espera esa confirmacion.

3. Conflictos y borrados

Para resolver conflictos entre actualizaciones, el sistema aplica una política de "última escritura gana"basándose en una comparación estricta de timestamps. La regla general es: una actualización o tombstone nueva con una marca menor no puede sobrescribir a una información más reciente; luego, una escritura o un ofrecimiento de clave-valor que llega con timestamp viejo (es decir menor al actual) no tiene la capacidad de resucitarüna tombstone. En el caso de que dos operaciones originadas por procesos distintos tengan exactamente el mismo microsegundo de timestamp, el PID del proceso actúa como un criterio de desempate determinista.

Es importante notar que intervienen funcionalidades operativas como el desalojo de claves (por Alfa), el cual elimina por completo tanto el valor como su timestamp asociada, permitiendo que una operación antigua que llegue con posterioridad sea aceptada por error. Esto genera estados transitorios donde una réplica puede conservar una tombstone mientras otra mantiene un valor activo, sumado a que una lectura puramente local no realiza comparaciones con las versiones que puedan tener las réplicas remotas.

4. Vencimiento

Un `put` con un tiempo de vida (TTL) finito programa un temporizador mediante `send_after(TTL, self(), {ttl_expired, Key})` directamente en la réplica que recibe la escritura original; si el TTL es `infinity`, no se programa ningún temporizador. Cuando se envia el mensaje de vencimiento y es procesado, la réplica genera una timestamp nueva, instala una tombstone para la clave y procede a propagarla al resto de réplicas. El proceso de vencimiento opera de forma asíncrona: no se verifica de manera bloqueante al momento de realizar una lectura y su ejecución puede verse ligeramente demorada si el buzón del actor esta ocupado. Una cosa a notar es que las réplicas que reciben un `silent_put` no almacenan el valor TTL, la unica que puede ejecutar el vencimiento es la replica original que guasrdo el valor, el resto depende de la señal de borrado difundida por esta réplica

Para evitar que una clave actualizada sea eliminada por el temporizador de vencimiento del anterior put de la misma clave, se asocia cada evento de expiración a la timestamp específica de la escritura que la generó. La réplica luego verifica que la versión actual de la clave coincida exactamente con la que programó el temporizador. Si la entrada fue sobrescrita localmente o recibió una actualización posterior antes de expirar, el temporizador no hace nada, se garantiza entonces la consistencia del ciclo de vida ante actualizaciones concurrentes.

5. Desalojo y terminación

Justo después de procesar una operación local de `put`, la función `enforce_alpha` evalúa el estado del almacenamiento contando solamente las entradas válidas (que no sean tombstones). Si el número excede el límite capacidad α, se selecciona el elemento con la timestamp mas vieja, luego se lo ofrece de forma individual a cada una de las demás réplicas con `offer_key_value` y lo elimina localmente, sin generar una tombstone. Luego, otra réplica solo acepta la oferta si tiene la capacidad disponible y la versión ofrecida es más reciente que la que tiene almacenada, o si directamente no tiene la clave localmente. En caso de que la réplica receptora esté llena, la clave ofrecida se descarta; el mecanismo no hace que la replica que ofrece el valor espere a una respuesta, la replica envia el mensaje de ofrecimiento y sigue con su ejecucion normal, no hay garantia de que se guarde localmente en alguna de las replicas. Notar que la política aplicada para el desalojo se basa en la antigüedad de la escritura (FIFO por timestamp), y no en una política de uso reciente (LRU).

6. Lectura distribuida

Si una clave se encuentra presente de forma local, `get` devuelve directamente su valor asociado o `not_found` en caso de que haya una tombstone en su lugar. Por otro lado cuando la clave no existe en la replica actual, esta consulta al resto de replicas, enviando una petición a los demás procesos. Por diseño, el valor obtenido de otra replica no se guarda automáticamente en el map local, ya que se asume que si no existe es porque se borro por Alfa, o la consistencia eventual todavia no le 'llego' a esa replica.

Para coordinar esta consulta de forma segura y evitar conflictos entre llamadas concurrentes, se utilizan referencias únicas (`QueryRef`). Cuando una replica emite la consulta hacia sus vecinos, utiliza mensajes específicos que no propagan recursivamente la búsqueda (`get_no_propagate`), rompiendo cualquier posibilidad de ciclos infinitos o deadlocks.

Del lado del recolector de respuestas, la espera interna cuenta con un límite acotado de $N - 1$ períodos de tiempo (con un temporizador de seguridad de 50 ms por cada vecino consultado). Tras cada respuesta negativa o vencimiento del temporizador, el contador decrece de forma controlada hasta finalizar. Luego entonces se previene una espera circular permanente entre réplicas.